



Edifiq Documentação Schema

Banco de dados: MySQL 8.0+ · Arquitetura: Multi-tenant SaaS com RBAC

Visão Geral

O schema é organizado em **7 domínios funcionais**:

1. SaaS & Planos
2. Usuários & Controle de Acesso
3. Fornecedores
4. Pedidos
5. Leilão Reverso
6. Pós-seleção
7. Infraestrutura & Auditoria

1. SaaS & Planos

plans

Define os planos de assinatura disponíveis na plataforma (free, starter, pro, enterprise).

Cada plano estabelece os limites operacionais de um tenant: quantos usuários podem ser cadastrados, quantos fornecedores podem ser gerenciados, quantos pedidos podem ser criados por mês e quais funcionalidades estão disponíveis (analytics, acesso à API). É a tabela que governa o modelo de negócio do SaaS.

Coluna	Descrição
name	Nome do plano (unique)

Coluna	Descrição
<code>max_users</code>	Limite de usuários por tenant
<code>max_suppliers</code>	Limite de fornecedores por tenant
<code>max_orders_per_month</code>	Limite de pedidos mensais
<code>has_analytics</code>	Acesso ao painel de analytics
<code>has_api_access</code>	Acesso à API pública
<code>price_monthly</code>	Preço mensal do plano

tenants

Representa cada empresa cliente que contratou o SaaS.

É a âncora de isolamento de dados: praticamente todas as outras tabelas possuem um `tenant_id` que garante que cada empresa só enxerga seus próprios dados. Um tenant nasce em status `trial` e avança conforme a assinatura. O campo `settings` em JSON permite configurações personalizadas por tenant sem alterar o schema (ex: nome da empresa exibido, fuso horário, notificações).

Coluna	Descrição
<code>plan_id</code>	Plano ativo do tenant
<code>slug</code>	Identificador único na URL (ex: <code>construtora-abc</code>)
<code>status</code>	Ciclo de vida: <code>trial</code> → <code>active</code> → <code>suspended</code> → <code>cancelled</code>
<code>settings</code>	Configurações personalizadas em JSON
<code>trial_ends_at</code>	Data de expiração do período gratuito

2. Usuários & Controle de Acesso

users

Armazena os usuários de cada tenant — engenheiros, compradores, gestores e administradores.

O par `(tenant_id, email)` é único, o que significa que o mesmo e-mail pode existir em tenants diferentes (empresas distintas), mas não duas vezes dentro da mesma

empresa. O campo `is_active` permite desativar um usuário sem excluí-lo, preservando o histórico de ações.

Coluna	Descrição
<code>tenant_id</code>	Empresa à qual o usuário pertence
<code>email</code>	Login do usuário (único por tenant)
<code>password_hash</code>	Senha criptografada
<code>is_active</code>	Permite suspender acesso sem deletar o registro
<code>email_verified</code>	Controle de verificação de e-mail no cadastro
<code>last_login_at</code>	Útil para detectar usuários inativos

roles

Define os perfis de permissão disponíveis dentro de cada tenant.

Implementa o modelo RBAC (Role-Based Access Control). As permissões ficam em um array JSON, o que permite adicionar novas permissões sem alterar o schema. O campo `is_system` marca roles protegidas criadas automaticamente (como `owner` e `admin`), que não podem ser editadas ou excluídas pelo tenant.

Coluna	Descrição
<code>tenant_id</code>	Roles são isoladas por tenant
<code>permissions</code>	Array JSON com as permissões (ex: <code>["orders.create", "proposals.view"]</code>)
<code>is_system</code>	Protege roles nativas de alterações indevidas

user_roles

Tabela de junção que vincula usuários a seus perfis de acesso.

Um usuário pode ter múltiplos roles (ex: `buyer` + `viewer`). O campo `granted_by` registra quem concedeu o acesso, criando uma trilha de responsabilidade para auditorias.

Coluna	Descrição
<code>user_id</code>	Usuário que recebe o perfil

Coluna	Descrição
<code>role_id</code>	Perfil concedido
<code>granted_by</code>	Usuário que realizou a concessão

3. Fornecedores

categories

Tabela global (não isolada por tenant) com as categorias de materiais de construção.

Suporta hierarquia via `parent_id` (ex: "Hidráulica" pode ter filhas como "Tubos", "Conexões", "Registros"). É usada tanto para classificar os itens dos pedidos quanto para definir quais tipos de materiais cada fornecedor está apto a atender, permitindo que o motor de distribuição faça o match correto.

Coluna	Descrição
<code>parent_id</code>	Permite subcategorias (hierarquia em árvore)
<code>slug</code>	Identificador amigável para uso em URLs e filtros

suppliers

Cadastro dos fornecedores (depósitos, lojas de materiais) que competem nos leilões.

Além dos dados cadastrais, armazena a geolocalização (`lat` , `lng`) que o motor de distribuição usa para filtrar fornecedores próximos ao endereço de entrega. O `reputation_score` é atualizado automaticamente via trigger sempre que uma nova avaliação é registrada. O `response_sla_min` define o tempo máximo que o fornecedor tem para enviar uma proposta antes de ser desconsiderado.

Coluna	Descrição
<code>tenant_id</code>	Fornecedores são gerenciados por tenant
<code>lat / lng</code>	Geolocalização para filtro por proximidade
<code>reputation_score</code>	Média das avaliações (0–5), atualizada automaticamente
<code>max_delivery_km</code>	Raio máximo de entrega do fornecedor

Coluna	Descrição
<code>response_sla_min</code>	Prazo para responder um pedido (em minutos)
<code>status</code>	<code>active</code> / <code>inactive</code> / <code>blocked</code>

supplier_categories

Tabela de junção que define quais categorias de materiais cada fornecedor atende.

É a peça central do motor de distribuição inteligente: quando um pedido chega com itens de determinadas categorias, o sistema consulta esta tabela para identificar quais fornecedores estão aptos a receber a notificação.

Coluna	Descrição
<code>supplier_id</code>	Fornecedor
<code>category_id</code>	Categoria que ele atende

4. Pedidos

orders

Representa a necessidade de materiais publicada pelo comprador — o ponto de partida do leilão reverso.

Quando um pedido é criado com status `draft`, ele ainda não está visível para fornecedores. Ao ser publicado (`open` → `in_auction`), o motor de orquestração é acionado. O campo `auction_duration_min` define por quanto tempo o leilão ficará aberto para receber propostas. A geolocalização do endereço de entrega é armazenada para cálculo de proximidade com fornecedores.

Coluna	Descrição
<code>status</code>	Ciclo de vida: <code>draft</code> → <code>open</code> → <code>in_auction</code> → <code>selected</code> → <code>confirmed</code> → <code>cancelled</code> / <code>expired</code>
<code>is_urgent</code>	Pedidos urgentes podem ter regras de distribuição diferenciadas
<code>delivery_lat</code> / <code>lng</code>	Usado pelo motor para calcular distância até os fornecedores
<code>max_suppliers</code>	Quantos fornecedores serão notificados no leilão

Coluna	Descrição
<code>auction_duration_min</code>	Janela de tempo para receber propostas
<code>reference_code</code>	Código interno da obra ou requisição
<code>metadata</code>	Dados extras flexíveis em JSON

order_items

Detalha os itens individuais de um pedido.

Um pedido pode ter múltiplos itens (ex: 20 sacos de cimento + 5 m³ de areia + 100 blocos). Cada item tem sua categoria, quantidade e unidade de medida, permitindo que os fornecedores citem cada produto separadamente na tabela

`proposal_items`.

Coluna	Descrição
<code>order_id</code>	Pedido ao qual o item pertence
<code>category_id</code>	Categoria do material (usado no match com fornecedores)
<code>description</code>	Descrição do material solicitado
<code>quantity</code>	Quantidade necessária
<code>unit</code>	Unidade de medida (un, kg, m ² , saco, m ³ etc.)
<code>sort_order</code>	Ordem de exibição dos itens

order_distributions

Registra quais fornecedores foram notificados para cada pedido e por qual canal.

Garante rastreabilidade total da distribuição: é possível saber exatamente quem foi notificado, quando, se abriu a notificação e por qual meio (e-mail, SMS, push, webhook). Evita notificações duplicadas e serve de base para métricas de engajamento dos fornecedores.

Coluna	Descrição
<code>order_id</code>	Pedido distribuído
<code>supplier_id</code>	Fornecedor notificado
<code>channel</code>	Canal de envio: <code>email</code> , <code>sms</code> , <code>push</code> , <code>webhook</code>

Coluna	Descrição
<code>opened_at</code>	Registro de quando o fornecedor visualizou a notificação

5. Leilão Reverso

`proposals`

Armazena as propostas que os fornecedores enviam em resposta a um pedido.

É o núcleo do leilão reverso. Cada fornecedor pode enviar uma proposta com seu preço total e prazo de entrega estimado. A constraint `UNIQUE (order_id, supplier_id)` garante que cada fornecedor envie apenas uma proposta por pedido (podendo atualizá-la enquanto o leilão estiver aberto). O `expires_at` define até quando a proposta é válida.

Coluna	Descrição
<code>status</code>	<code>pending</code> → <code>submitted</code> → <code>accepted</code> / <code>rejected</code> / <code>expired</code> / <code>withdrawn</code>
<code>total_price</code>	Valor total ofertado pelo fornecedor
<code>delivery_min</code>	Prazo de entrega estimado em minutos
<code>expires_at</code>	Validade da proposta

`proposal_items`

Detalha a cotação de cada item individualmente dentro de uma proposta.

Permite que o comprador veja não só o preço total, mas o preço unitário de cada material. O campo `available` indica se o fornecedor possui aquele item em estoque — um fornecedor pode enviar uma proposta mesmo que não tenha todos os itens, deixando `available = false` nos que não consegue atender.

Coluna	Descrição
<code>proposal_id</code>	Proposta à qual o item pertence
<code>order_item_id</code>	Item do pedido sendo cotado
<code>unit_price</code>	Preço unitário ofertado
<code>quantity</code>	Quantidade que o fornecedor consegue entregar
<code>available</code>	Se o item está disponível em estoque

6. Pós-seleção

order_selections

Registra a decisão final do comprador ao escolher uma proposta, encerrando o leilão.

A constraint `UNIQUE (order_id)` garante que um pedido só pode ter uma seleção — uma vez escolhido o fornecedor, o leilão está encerrado. O campo `reason` permite que o comprador justifique a escolha (ex: "melhor prazo" ou "melhor preço"), gerando dados para análises futuras.

Coluna	Descrição
<code>order_id</code>	Pedido encerrado (único)
<code>proposal_id</code>	Proposta vencedora
<code>selected_by</code>	Usuário que tomou a decisão
<code>reason</code>	Justificativa da escolha

deliveries

Acompanha o ciclo de vida da entrega após a seleção do fornecedor.

Cada seleção gera exatamente uma entrega. Registra todas as etapas: agendamento, despacho, conclusão ou falha. O `proof_url` armazena a URL de um documento ou foto que comprova a entrega, útil para resolução de disputas.

Coluna	Descrição
<code>status</code>	<code>scheduled</code> → <code>in_transit</code> → <code>delivered</code> / <code>failed</code> / <code>returned</code>
<code>tracking_code</code>	Código de rastreamento fornecido pelo entregador
<code>scheduled_at</code>	Data/hora combinada para a entrega
<code>dispatched_at</code>	Momento em que saiu para entrega
<code>delivered_at</code>	Confirmação de recebimento
<code>proof_url</code>	Comprovante de entrega (foto, assinatura, NF)

ratings

Permite que o comprador avalie o fornecedor após a entrega.

As avaliações alimentam o `reputation_score` da tabela `suppliers` via trigger automático, criando um sistema de reputação dinâmico. O fornecedor pode registrar uma `response` à avaliação. A constraint `UNIQUE (order_selection_id, rated_by)` impede avaliações duplicadas para o mesmo pedido.

Coluna	Descrição
<code>score</code>	Nota de 1 a 5
<code>comment</code>	Comentário do comprador
<code>response</code>	Resposta do fornecedor à avaliação

7. Infraestrutura & Auditoria

webhooks

Cadastra endpoints externos para receber notificações de eventos da plataforma.

Permite que os tenants integrem o sistema com ERPs, sistemas de compras ou qualquer serviço externo. O campo `events` em JSON define quais eventos disparam o webhook (ex: `order.created`, `proposal.submitted`, `delivery.confirmed`). O `secret` é usado para assinar os payloads enviados, garantindo autenticidade.

Coluna	Descrição
<code>url</code>	Endpoint que receberá os eventos
<code>events</code>	Lista de eventos que disparam o webhook
<code>secret</code>	Chave para assinatura HMAC dos payloads
<code>is_active</code>	Liga/desliga o webhook sem excluí-lo

webhook_deliveries

Histórico de cada disparo de webhook realizado pela plataforma.

Essencial para debugging e observabilidade. Registra o payload enviado, o status HTTP retornado e a resposta recebida. Permite implementar retry automático nos casos em que a entrega falhou (status diferente de 2xx).

Coluna	Descrição
<code>webhook_id</code>	Webhook que originou o disparo
<code>event</code>	Nome do evento disparado
<code>payload</code>	Corpo da requisição enviada
<code>status_code</code>	HTTP status retornado pelo endpoint
<code>response</code>	Corpo da resposta recebida
<code>delivered_at</code>	Momento da entrega bem-sucedida

audit_logs

Registro imutável de todas as ações relevantes realizadas na plataforma.

Nenhum registro é atualizado ou deletado aqui — é apenas append. Captura quem fez o quê, em qual entidade, com qual payload, de qual IP e navegador. Fundamental para compliance, investigação de incidentes e rastreabilidade em ambientes B2B. O campo `entity` + `entity_id` permite reconstituir o histórico completo de qualquer registro.

Coluna	Descrição
<code>action</code>	Ação realizada (ex: <code>order.published</code> , <code>proposal.accepted</code>)
<code>entity</code>	Tabela afetada (ex: <code>orders</code> , <code>proposals</code>)
<code>entity_id</code>	ID do registro afetado
<code>payload</code>	Estado antes/depois ou dados da ação em JSON
<code>ip_address</code>	IP de origem da requisição
<code>user_agent</code>	Navegador/cliente que originou a ação

Fluxo Completo Resumido

```

plans → tenants → users → roles → user_roles
      ↓
      orders → order_items
            ↓
order_distributions → suppliers ← supplier_categories ← categories
            ↓
      proposals → proposal_items
            ↓

```

```
order_selections
  ↓
  deliveries + ratings → (atualiza suppliers.reputation_score)
    ↓
webhooks → webhook_deliveries
audit_logs (registra tudo)
```